

Week-7 Practical

1. Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

```
imvarls@LAPTOP-VQ4D70L7: × + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>

int global_var = 10;
static int static_var = 20;

void code_function() {
    printf("Code address    : %p\n", (void *)code_function);
}

int main() {
    int stack_var = 30;
    int *heap_var = malloc(sizeof(int));
    *heap_var = 40;

    printf("Memory Addresses\n");
    printf("-----\n");

    code_function();
    printf("Global address : %p\n", (void *)&global_var);
    printf("Static address : %p\n", (void *)&static_var);
    printf("Heap address   : %p\n", (void *)heap_var);
    printf("Stack address  : %p\n", (void *)&stack_var);

    free(heap_var);
    return 0;
}
```

Output:

```
imvarls@LAPTOP-VQ4D70L7: × + v
imvarls@LAPTOP-VQ4D70L7:~$ nano memory.c
imvarls@LAPTOP-VQ4D70L7:~$ gcc memory.c -o memory
imvarls@LAPTOP-VQ4D70L7:~$ ./memory
Memory Addresses
-----
Code address    : 0x5baf6391d1c9
Global address  : 0x5baf63920010
Static address  : 0x5baf63920014
Heap address    : 0x5baf6acf9010
Stack address   : 0x7ffc329176c
imvarls@LAPTOP-VQ4D70L7:~$ |
```

2. Make use of the `/proc//maps` file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

```
imvarls@LAPTOP-VQ4D70L7: × + v
GNU nano 8.7.1 memory_m
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 100;

int main() {
    int stack_var = 200;
    int *heap_var = malloc(sizeof(int));
    *heap_var = 300;

    printf("Process PID: %d\n", getpid());
    printf("Global address: %p\n", (void *)&global_var);
    printf("Stack address : %p\n", (void *)&stack_var);
    printf("Heap address  : %p\n", (void *)heap_var);

    printf("\nProcess is running...\n");
    printf("Use another terminal to inspect /proc/%d/maps\n", getpid());

    while (1) {
        sleep(1);
    }

    free(heap_var);
    return 0;
}
```

Output:

```
imvarls@LAPTOP-VQ4D70L7: × + v
imvarls@LAPTOP-VQ4D70L7:~$ nano memory_maps.c
imvarls@LAPTOP-VQ4D70L7:~$ gcc memory_maps.c -o memory_maps
imvarls@LAPTOP-VQ4D70L7:~$ ./memory_maps
Process PID: 906
Global address: 0x5d494db09010
Stack address : 0x7ffc4b41d1dc
Heap address  : 0x5d497214f010

Process is running...
Use another terminal to inspect /proc/906/maps
|
```

imvarls@LAPTOP-VQ4D70L7: x + v

```
imvarls@LAPTOP-VQ4D70L7:~$ grep heap /proc/906/maps
```

```
5d497214f000-5d4972170000 rw-p 00000000 00:00 0
```

[heap]

```
imvarls@LAPTOP-VQ4D70L7:~$ grep stack /proc/906/maps
```

```
7ffc4b3fe000-7ffc4b41f000 rw-p 00000000 00:00 0
```

[stack]

```
imvarls@LAPTOP-VQ4D70L7:~$ pmap 906
```

```
906: ./memory_maps
```

```
00005d494db05000      4K r---- memory_maps
00005d494db06000      4K r-x-- memory_maps
00005d494db07000      4K r---- memory_maps
00005d494db08000      4K r---- memory_maps
00005d494db09000      4K rw--- memory_maps
00005d497214f000    132K rw--- [ anon ]
00007ed338800000     160K r---- libc.so.6
00007ed338828000   1632K r-x-- libc.so.6
00007ed3389c0000    312K r---- libc.so.6
00007ed338a0e000     16K r---- libc.so.6
00007ed338a12000      8K rw--- libc.so.6
00007ed338a14000     52K rw--- [ anon ]
00007ed338bc5000     12K rw--- [ anon ]
00007ed338bcd000      8K rw--- [ anon ]
00007ed338bcf000     16K r---- [ anon ]
00007ed338bd3000      8K r---- [ anon ]
00007ed338bd5000      8K r-x-- [ anon ]
00007ed338bd7000      4K r---- ld-linux-x86-64.so.2
00007ed338bd8000    188K r-x-- ld-linux-x86-64.so.2
00007ed338c07000     44K r---- ld-linux-x86-64.so.2
00007ed338c12000      8K r---- ld-linux-x86-64.so.2
00007ed338c14000      4K rw--- ld-linux-x86-64.so.2
00007ed338c15000      4K rw--- [ anon ]
00007ffc4b3fe000    132K rw--- [ stack ]
total                2768K
```

```
imvarls@LAPTOP-VQ4D70L7:~$ |
```